Multimodal RAG Finance Product Implementation Prompt

Step by step execution plan and coding agent instructions

Purpose. Use this document as the source of truth for turning the existing working prototype into a complete, reliable product. The agent must improve answer quality, retrieval, extraction, numerical reasoning, evaluation, and the user interface without breaking the verified offline workflow.

The project already runs end to end on a synthetic PDF. The next job is to make it accurate on real reports, easy to operate, measurable, and polished enough for a final-year project demonstration.

How to use this document

1. Attach this document to the coding agent together with the repository.
2. Tell the agent to treat this document as the implementation brief and to execute the phases in order.
3. Require the agent to run each acceptance check before marking a phase complete.
4. Do not accept invented metrics, placeholder screens, fake progress, or untested claims.
5. Keep the existing offline demo working throughout the implementation.

Current project state

Area	Current state	Next required state
Pipeline	PDF to extraction to retrieval to grounded answer works	Production models and real reports
Ingestion	Text tables figures and OCR interface implemented	Real OCR and stronger table and figure detection
Search	Dense BM25 RRF and weighted fusion implemented	Pretrained encoders reranking and tuned retrieval
Generation	Citation gate and deterministic MockLLM work	Real provider quality streaming and stronger prompts
Evaluation	Synthetic three question smoke run exists	Verified 60 plus question real corpus benchmark
Frontend	React evidence search page and Streamlit client exist	Complete multi page product experience
Quality	28 Python tests 3 frontend tests 81 percent core coverage	Expanded regression E2E and accessibility coverage

Important limitation. The current measured result proves that the software runs. It does not prove real financial-document accuracy because it uses one synthetic report, three unverified questions, MockLLM, and the offline test encoder.

Master agent instruction

You are the senior ML platform and product engineer responsible for completing this repository. Work directly in the existing project. Preserve working behavior, implement each phase below, test it, record measured results, and leave the codebase ready for another team member to run without hidden knowledge.

Non negotiable rules

- Read the repository README configs architecture tests deviations and current reports before changing code.
- Use current official documentation before coding against fast moving library APIs.
- Do not train or fine tune representation models. Use pretrained encoders only.
- Keep text and table embeddings separate from CLIP image embeddings. Never compare their raw similarity scores directly.
- Keep citation metadata on every chunk and validate every generated citation against retrieved evidence.
- Keep the text only baseline fair. Change only table flattening and figure removal.
- Keep MockLLM and deterministic test encoders for offline CI. Never silently fall back to them in production mode.
- Do not hard code metrics or UI results. Every displayed number must come from a saved run artifact.
- Do not implement chart to data reconstruction. Static figures caption text surrounding text and OCR remain in scope.
- Add a regression test for each bug fixed and record meaningful deviations in docs deviations.

Required execution order

Phase	Outcome	Do not proceed until
0	Audit and safe baseline	Existing tests and fixture workflow pass
1	Real environment and corpus	Tesseract models and reports are validated
2	Answer quality	Real provider answers pass citation gate
3	Search quality	Retrieval improves on fixed gold questions
4	Table extraction	Representative financial tables are correct
5	Figure extraction	Charts are separated from logos and page scans
6	Numerical answers	Deterministic calculations and units are verified
7	Complete frontend	All product pages and states work in browser
8	Evaluation	Real baseline comparison and ablations are saved
9	Release audit	Clean setup tests docs Docker and demo pass

Phase 0 Audit and protect the working baseline

[ ] Run Python tests lint formatting and strict type checks.

[ ] Run frontend component tests and the production build.

[ ] Generate the synthetic fixture ingest it build both indexes ask one text one table and one figure question and run evaluation.

[ ] Save the current outputs as the before baseline. Do not overwrite old run artifacts.

[ ] Review configuration paths environment variables and model selection behavior.

[ ] Confirm that production mode fails clearly when dependencies credentials or model weights are missing.

```
uv sync --extra dev --extra app --extra eval --extra tables
uv run pytest
uv run ruff check .
uv run ruff format --check .
uv run mypy src/mmrag
npm test --prefix frontend
npm run build --prefix frontend
```

Phase 1 Prepare the real environment and corpus

Install and verify required software

[ ] Install Tesseract OCR and verify tesseract --version works from the same terminal used by the app.

[ ] Install the models extra and verify Sentence Transformers and OpenCLIP can load on CPU.

[ ] Choose one real generation provider: Anthropic OpenAI compatible endpoint or local Ollama.

[ ] Copy .env.example to .env and store keys locally. Never commit .env.

[ ] Record exact resolved package model and checkpoint versions in an ADR and lockfile.

```
uv sync --extra models --extra ocr --extra tables --extra app --extra eval --extra dev
tesseract --version
Copy-Item .env.example .env
```

Collect and validate documents

[ ] Download the 15 public annual reports listed in data manifests corpus.csv or update broken URLs with verified official sources.

[ ] Keep a mix of NSE BSE and NASDAQ companies and FY2022 to FY2024 reports.

[ ] Add at least one real scanned or image heavy report to exercise OCR.

[ ] Record retrieval date source URL checksum page count and terms for every report.

[ ] Reject HTML error pages renamed as PDF and skip corrupt or encrypted files with a logged reason.

[ ] Run ingestion and manually inspect several text pages tables figures and OCR pages before indexing the full corpus.

Phase 2 Improve answer quality

Goal. Produce clear useful answers that remain strictly grounded in retrieved evidence. Better writing must never weaken citation validation or abstention.

Implementation instructions

[ ] Add provider configuration for model name endpoint key environment variable timeout retry count max tokens and vision capability.

[ ] Add a provider health check that reports missing keys unavailable local models and endpoint failures without exposing secrets.

[ ] Version the generation prompt and save its hash in every answer and evaluation record.

[ ] Structure the prompt into question evidence rules calculation rules and output format.

[ ] Require citations after each factual claim rather than a citation only at the end.

[ ] Require an explicit statement of units currency scale period and any assumptions.

[ ] Add a compact answer style for simple questions and a detailed style for comparisons and calculations.

[ ] Support streaming from providers but buffer and validate the final answer before marking it grounded.

[ ] Allow a vision capable provider to receive only retrieved figure images and their text. Never pass unrelated pages.

[ ] Keep one strict retry. If validation still fails return Insufficient evidence and show the evidence.

[ ] Log latency token use provider model prompt version retry count and grounding outcome.

Acceptance checks

[ ] A real provider answers representative text table figure and multimodal questions.

[ ] Every accepted answer has only citations that resolve to retrieved chunk IDs.

[ ] An intentionally uncited or invalidly cited provider response is rejected after one retry.

[ ] Missing credentials produce an actionable error and never switch to MockLLM silently.

[ ] Answer quality is compared using a fixed rubric and at least 20 human spot checks.

Phase 3 Improve search quality

Production encoders

[ ] Use sentence transformers all MiniLM L6 v2 as the initial text and table encoder and normalized float32 embeddings.

[ ] Use OpenCLIP ViT B 32 for figure image embeddings and its text tower for figure queries.

[ ] Persist encoder IDs checkpoint IDs dimensions normalization and config hash with each index.

[ ] Refuse to load an index built with a different encoder or preprocessing configuration.

[ ] Cache embeddings by content hash and model identity so unchanged chunks are not re embedded.

Retrieval improvements

[ ] Tune candidate K final K and RRF constant on a development set without using test labels.

[ ] Benchmark dense only BM25 only RRF and weighted fusion on identical questions.

[ ] Add an optional cross encoder reranker for text and serialized tables behind a configuration flag.

[ ] Add query normalization for fiscal years abbreviations company names percentages and currencies.

[ ] Add conservative query expansion for known financial synonyms such as sales and revenue. Preserve the original query and log expansions.

[ ] Add document modality and date filters without changing the shared Retriever interface.

[ ] Evaluate a modality diversity rule and keep it only if it improves figure or table recall without harming overall quality.

[ ] Return an explanation object containing ranks source scores fusion contribution and filters for debugging and UI display.

Search acceptance checks

[ ] Text questions retrieve the correct text chunk in top five on the verified set.

[ ] Table questions retrieve the correct table chunk rather than only duplicate page text.

[ ] Figure questions retrieve the correct linked figure using both image and caption OCR signals.

[ ] Index save and load returns identical rankings for deterministic queries.

[ ] Measured latency and memory use are recorded for CPU operation.

Phase 4 Improve table extraction

Goal. Preserve the table structure that financial questions depend on, including row labels column labels units currency scale negative values and multi page continuation.

[ ] Run both PyMuPDF table detection and pdfplumber strategies on representative pages.

[ ] Create a table quality score using nonempty cell ratio header consistency column alignment row count repeated values and extraction warnings.

[ ] Select the better extractor per table or document and record the chosen extractor and score.

[ ] Detect titles and units from nearby text above and below the table. Store the source text and distance used.

[ ] Normalize merged or blank header cells without inventing missing values.

[ ] Support two row headers multi row column headers repeated headers and continued tables across adjacent pages.

[ ] Preserve exact display strings and also store normalized numeric values separately for calculations.

[ ] Serialize every data cell with explicit row and column labels plus title units and page.

[ ] Prevent duplicate table text from unfairly dominating retrieval by linking page text and table chunks.

[ ] Add visual debug output that draws table bounding boxes on rendered page images.

Table test cases

- Currency values with commas decimals and symbols.

- Percentages and basis points.
- Negative values shown with parentheses.
- Values with units in thousands millions billions lakhs and crores.
- Merged cells blank cells and repeated header rows.
- Tables split across two pages.

Phase 5 Improve figure extraction

Goal. Retrieve real charts and diagrams while filtering logos decorative images icons and full page scans that should not be treated as figures.

[ ] Classify extracted images using size aspect ratio location repetition across pages color entropy and caption proximity.

[ ] Reject repeated logos headers footers tiny icons separators and near blank images.

[ ] Treat a full page raster as an OCR page first. Create a figure only when it contains a meaningful chart or diagram region.

[ ] Extract vector chart regions by page rendering and region cropping when no embedded image exists.

[ ] Link the nearest Figure Chart Exhibit or Source caption using bounding box distance and reading order.

[ ] Capture surrounding paragraph text and OCR text inside each image.

[ ] Store image caption OCR surrounding text bounding box extraction method and quality score on the same figure chunk.

[ ] Index the image in the CLIP space and all linked text in text dense and BM25 spaces under the same chunk ID.

[ ] Add a figure review gallery that shows thumbnail page caption OCR and filter decision.

[ ] Keep chart to data reconstruction out of scope. Do not claim exact hidden series values from visual estimation.

Figure acceptance checks

[ ] A chart heavy sample retrieves the expected chart for at least ten verified figure questions.

[ ] Repeated company logos do not appear in normal top K evidence.

[ ] Every displayed figure endpoint resolves safely to a file inside the processed data directory.

[ ] Scanned pages remain available for OCR and page preview without being mislabeled as empty figures.

Phase 6 Improve numerical answers

Goal. Move arithmetic and unit handling out of free form language generation when the retrieved evidence contains structured numbers.

[ ] Create a typed financial value parser for currency symbol numeric value scale unit period row label and column label.

[ ] Support commas decimals percentages parentheses for negatives and common Indian and international scales.

[ ] Create deterministic operations for difference percentage change ratio sum average minimum and maximum.

[ ] Require the calculation planner to reference retrieved table cell IDs or exact source fields.

[ ] Execute arithmetic in Python Decimal rather than asking the LLM to calculate.

[ ] Return an arithmetic trace containing operands formula intermediate values final unit and citations.

[ ] Reject calculations when units periods or scales are incompatible or ambiguous.

[ ] Let the LLM explain the verified calculation but never replace the computed result.

[ ] Add numerical tolerance tests and ensure zero divide missing data and sign errors are handled.

Required numerical answer format

```
Answer: Revenue increased by 13.6 percent.
Calculation: (1250 - 1100) / 1100 x 100 = 13.6 percent.
Units: USD millions.
Evidence: [document-id, p.12, table]
```

Phase 7 Build a complete product frontend

Keep FastAPI as the backend boundary and React TypeScript as the main product. Keep Streamlit as a small compatibility client. The React app must work fully in demo mode and degrade clearly when real services are unavailable.

Product navigation

- Search workspace
- Document library
- Evaluation results
- Saved history
- Settings

Document library page

[ ] List every document with filename company fiscal year pages processing state text chunks tables figures OCR pages warnings and last indexed time.

[ ] Add search sort filter and status chips for ready processing failed and needs review.

[ ] Open a document detail view with ingestion report page thumbnails extracted tables figure gallery and warnings.

[ ] Allow safe reingestion reindex and removal with confirmation. Never delete the original file without an explicit separate action.

Upload and ingestion progress

[ ] Validate PDF MIME type extension size signature encryption and duplicate checksum before saving.

[ ] Create a background job with states queued validating extracting OCR tables figures indexing complete and failed.

[ ] Expose typed job endpoints and use server sent events WebSocket or polling for progress.

[ ] Show page level progress elapsed time current stage warning count and a cancel action.

[ ] Keep partial artifacts isolated until the job completes and make retries idempotent.

[ ] Show actionable errors such as Tesseract missing corrupt PDF model unavailable and disk failure.

Evaluation results page

[ ] Read only real report artifacts from reports run id. Never place sample numbers directly in frontend code.

[ ] Show baseline versus multimodal Recall at K MRR precision citation validity groundedness numeric match latency and confidence intervals.

[ ] Add overall per question type per modality and ablation tabs.

[ ] Show sample size verified question count configuration model IDs run time and git SHA.

[ ] Add a failure explorer that displays question reference answer system answer evidence and error category.

Full PDF page drawer with bounding box highlights

[ ] Open the original rendered PDF page from any citation or evidence card.

[ ] Overlay the chunk bounding box using the page coordinate system and account for scaling rotation and responsive resizing.

[ ] Support zoom pan next and previous page and show the exact document page modality and chunk ID.

[ ] Scroll the evidence card into view when a citation is clicked and visually highlight both the card and page region.

[ ] Handle chunks without a bounding box by showing the page with a clear no region available message.

Settings for retrieval and providers

[ ] Expose final K candidate K fusion method RRF constant dense weight sparse weight modalities document scope provider model vision and demo mode.

[ ] Validate incompatible combinations and explain their effect in plain language.

[ ] Keep secrets on the server. The browser may select a configured provider but must never receive API keys.

[ ] Show active model IDs index build version and whether settings require reindexing.

[ ] Allow reset to safe defaults and include the chosen settings with each request and saved history entry.

Better mobile layout

[ ] Design for 360 pixel width without horizontal page scrolling.

[ ] Use a bottom sheet or full screen route for evidence and PDF viewing on phones.

[ ] Make touch targets at least 44 pixels and keep keyboard focus visible.

[ ] Collapse advanced scores and settings by default while keeping citations readable.

[ ] Test common phone tablet laptop and large desktop viewports in Playwright.

Saved question history

[ ] Store question answer citations settings document scope timestamp latency and grounding outcome.

[ ] Use SQLite for durable local history or IndexedDB only for a clearly labeled browser local mode.

[ ] Support search rename pin rerun duplicate and delete with confirmation.

[ ] Do not store provider keys prompts with secrets or raw uploaded files in history.

[ ] Allow export as JSON or Markdown and provide a clear all history deletion action.

Additional product completion requirements

[ ] Add side by side multimodal versus baseline comparison to the React app.

[ ] Add empty loading skeleton success warning insufficient evidence offline and failure states.

[ ] Add keyboard navigation ARIA labels visible focus contrast and reduced motion support.

[ ] Add request cancellation retry and timeouts. Prevent duplicate submissions.

[ ] Add structured backend logs request IDs health checks and a small diagnostics screen.

[ ] Add secure path validation upload size limits safe filenames CORS configuration and no secret logging.

[ ] Keep the research prototype and not financial advice disclaimer visible.

Phase 8 Improve evaluation

Evaluation is the evidence that improvements are real. Do not tune on the final test questions and do not publish unverified machine generated questions as truth.

Gold data

[ ] Create at least 60 own corpus questions balanced across text table figure and multimodal types.

[ ] Record reference answer gold document page modality chunk IDs units question type author reviewer and verified flag.

[ ] Require a human to open the cited page and approve every headline question.

[ ] Keep a development split for tuning and a locked test split for final reporting.

[ ] Load official FinQA and TAT QA contexts and labels for financial table and text evaluation.

[ ] Use a small seeded DocVQA and SlideVQA subset for generalisation and label it as a smaller check.

Metrics and comparisons

[ ] Report Recall at 1 3 5 and 10 Precision at 1 3 5 and 10 and MRR.

[ ] Report citation validity groundedness answer relevance numeric exact match and latency.

[ ] Report overall and per question type per modality and verified subset.

[ ] Run baseline versus multimodal dense only versus BM25 only versus hybrid RRF versus weighted and text to tables to figures ablations.

[ ] Add bootstrap 95 percent confidence intervals and a paired significance test.

[ ] Spot check at least 20 LLM judge decisions and report agreement with human reviewers.

[ ] Save config raw per question results metrics summary plots environment model IDs and git SHA under a unique run ID.

[ ] Create a regression threshold file so CI warns when verified fixture or gold metrics drop beyond an allowed tolerance.

Failure analysis

- Wrong extraction
- Correct chunk ranked too low
- Correct evidence but wrong arithmetic
- Unsupported answer or invalid citation
- Ambiguous question or incomplete gold label
- Model or provider failure
- Out of scope visual interpretation

Phase 9 Testing security and release readiness

Backend tests

[ ] Unit tests for value parsing table scoring figure filtering fusion reranking prompt rendering citation validation and job state transitions.

[ ] Integration tests for real style fixture PDFs including digital scanned multi page table and vector chart cases.

[ ] API tests for uploads progress documents page rendering figure serving settings history and evaluation reports.

[ ] End to end tests with MockLLM that cover grounded insufficient evidence invalid citation baseline and multimodal paths.

[ ] Keep CI network free GPU free deterministic and at or above 80 percent core line coverage.

Frontend tests

[ ] Component tests for citations evidence filters table rendering progress states settings history charts and PDF overlays.

[ ] Playwright flows for text table and figure questions upload document compare mode source drawer evaluation page settings dark mode mobile view and history.

[ ] Fail tests on console errors failed network requests unhandled exceptions inaccessible controls and broken images.

[ ] Save final light dark desktop and mobile screenshots under docs img and review spacing hierarchy readability and empty states twice.

Security checks

[ ] Never expose API keys to the frontend or logs.

[ ] Resolve and validate all requested document figure and page paths inside approved data roots.

[ ] Sanitize upload filenames enforce size and page limits and reject non PDFs.

[ ] Use parameterized SQLite queries and validate every request with typed schemas.

[ ] Document whether external providers receive extracted financial evidence and provide a local only option.

Phase 10 Final verification and handoff

[ ] Verify clean setup from a fresh environment using the documented commands.

[ ] Run fixture ingestion indexing evaluation API React Streamlit Docker and browser tests.

[ ] Run the full real corpus and record ingestion indexing and per query latency on the test machine.

[ ] Update README architecture evaluation results deviations gold review and all relevant ADRs.

[ ] Update the final report and presentation only with measured run IDs and real screenshots.

[ ] List remaining manual downloads human reviews credentials licences and environment constraints.

[ ] Produce a final handoff containing what changed measured results exact commands limitations and human actions.

Required commands for the finished project

```
# Install
uv sync --extra models --extra ocr --extra tables --extra app --extra eval --extra dev
npm install --prefix frontend

# Prepare and process data
uv run python scripts/download_corpus.py
uv run mmrag ingest data/raw --force
uv run mmrag index
```

```
uv run mmrag index --baseline

# Evaluate
uv run mmrag eval --gold data/eval/own_corpus_gold.jsonl

# Run product
make app

# Verify
uv run pytest
uv run ruff check .
uv run ruff format --check .
uv run mypy src/mmrag
npm test --prefix frontend
npm run build --prefix frontend
```

Definition of done

[ ] Fifteen public reports are traceable through the manifest and processed successfully or have a documented manual action.

[ ] OCR works on a real scanned page and reports confidence and language.

[ ] Production text table and figure encoders build and reload versioned indexes.

[ ] A real LLM or local model produces citation validated answers and abstains correctly.

[ ] Numerical answers use deterministic arithmetic and show formulas units and evidence.

[ ] The document library upload progress evaluation page PDF highlight drawer settings mobile layout and saved history work in React.

[ ] Baseline and multimodal systems are evaluated on identical human verified questions with confidence intervals ablations and failure analysis.

[ ] All tests lint typing builds Docker and browser checks pass from a clean setup.

[ ] No fake metrics placeholders TODO statements secrets or undocumented deviations remain in delivered paths.

Agent reporting format

At each milestone, report only the following:

- What was implemented
- Which files changed
- Which commands were run and their result
- Measured output or acceptance evidence
- Known limitations or next blocker

Do not claim a milestone is complete until its acceptance checks pass. If real documents models or credentials are unavailable, complete all offline work, record the exact missing input, and leave one clear command for a human to continue.